

Capítulo 1

Knowledge Base & Bitácora de Aprendizaje

Este capítulo constituye el repositorio central de conocimientos técnicos, conceptos teóricos y metodologías asimiladas durante auditorías, resolución de laboratorios y estudio bibliográfico. A diferencia de los reportes individuales, aquí la información se estructura de forma **temática y transversal**, vinculando cada habilidad con los libros, sitios web, documentación y máquinas específicas donde fue adquirida.

1.1. Metodología de Registro y Fuentes de Consulta

Para que el conocimiento perdure y sea consultable de manera inmediata en auditorías reales o exámenes prácticos (como OSCP o CPTS), cada concepto registrado debe responder a tres preguntas fundamentales:

1. **¿Qué aprendí?** (Concepto técnico, mecanismo de funcionamiento, payload o comando clave).
2. **¿Dónde lo aprendí?** (Libro con autor y capítulo, sitio web/documentación con URL, o máquina vulnerable con enlace interno).
3. **¿Cómo se aplica?** (Comando reproducible, script o prueba de concepto).

Badges de Fuentes Disponibles

Para estandarizar el origen del conocimiento, dispones de la macro `\sourceBadge{. . . }` y el comando `\resourceItem{Tipo}{Título/Autor}{Detalle/Enlace}`:



1.2. Fundamentos de Lenguajes & Arquitectura

1.2.1. Assembly (ASM x86_64) & Arquitectura de Computadores

El conocimiento de ensamblador x86_64 es fundamental para tareas de ingeniería inversa, depuración de binarios (*pwn*) y desarrollo de exploits.

Conceptos Clave Aprendidos: Arquitectura x86_64

- **Registros de Propósito General:** Familiarización con los registros de 64 bits (rax, rbx, rcx, rdx, rsi, rdi, rsp, rbp, r8–r15) y sus sub-registros de 32, 16 y 8 bits (eax, ax, al).
- **Convenio de Llamadas (System V AMD64 ABI):** Los primeros seis argumentos enteros/punteros a funciones en Linux se pasan a través de registros en orden estricto: rdi, rsi, rdx, rcx, r8, r9. Los argumentos adicionales van a la pila (*stack*).
- **Llamadas al Sistema (syscall):** El número de syscall se coloca en rax, los argumentos en rdi, rsi, etc., y el valor de retorno se recupera en rax.
- **Estructura del Stack Frame:** Creación de prólogo (push rbp; mov rbp, rsp) y epílogo (leave; ret) y reserva de variables locales restando a rsp.

Fuentes y Bibliografía Utilizadas

-  **Libro** Practical Binary Analysis (Dennis Andriesse)
[Capítulo 2: x86 and x86-64 Architecture Overview \(No Starch Press\)](#)
-  **Libro** Assembly Language Step-by-Step (Jeff Duntemann)
[Fundamentos de registros, banderas de estado y direccionamiento de memoria](#)
-  **Docs** x86-64 Cheat Sheet — Brown University
https://cs.brown.edu/courses/cs033/docs/guides/x64_cheatsheet.pdf
-  **Web** Linux System Call Table for x86_64 (Ray Chapman)
https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/
-  **Docs** Intel 64 and IA-32 Architectures Software Developer's Manual
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

</> Ejemplo Práctico: Syscall sys_write en x86_64 (hello.asm)

```
global _start

section .text
_start:
    ; syscall: sys_write (rax = 1, rdi = fd 1, rsi = buffer, rdx = len)
    mov rax, 1          ; ID de syscall sys_write en Linux x86_64
    mov rdi, 1          ; File Descriptor: 1 (stdout)
    mov rsi, msg         ; Puntero al mensaje
    mov rdx, len         ; Longitud en bytes
    syscall

    ; syscall: sys_exit (rax = 60, rdi = status code 0)
    mov rax, 60
    xor rdi, rdi
    syscall

section .data
msg db "Hello, x86_64 Assembly!", 0x0A
len equ $ - msg
```

1.2.2. Bash Scripting & Automatización de Tareas

Bash es el lenguaje nativo de los entornos Linux para automatización rápida de auditorías, parsing de ficheros de log y creación de herramientas de reconocimiento al vuelo.

** Conceptos Clave Aprendidos: Bash Avanzado**

- **Redirección y Descriptores de Ficheros:** Control preciso de stdin (0), stdout (1) y stderr (2). Uso de 2>&1 para unificar flujos y /dev/null para silenciar ruido en bucles.
- **One-Liners de Reconocimiento:** Uso de combinaciones de curl, grep, sed, awk y xargs para extraer subdominios o endpoints activos en segundos sin herramientas pesadas.
- **Subshells y Captura de Salida:** Diferencia entre asignación síncrona mediante \$(comando) y ejecución en subshell background con & junto al comando wait.
- **Manejo de Señales (trap):** Limpieza automática de ficheros temporales y sockets al pulsar Ctrl+C (SIGINT) en scripts interactivos.

Fuentes y Bibliografía Utilizadas

-  **Libro** **The Linux Command Line: A Complete Introduction (William Shotts)**
[Capítulos 20 al 35: Entornos shell, variables, control de flujo y scripts robustos](#)
-  **Libro** **Wicked Cool Shell Scripts (Dave Taylor & Brandon Perry)**
[2ª Edición — Scripts prácticos para administración de sistemas y seguridad](#)
-  **Web** **Bash Hackers Wiki — Guía profunda de sintaxis y buenas prácticas**
<https://wiki.bash-hackers.org>
-  **Web** **ExplainShell — Análisis visual e interactivo de comandos y flags**
<https://explainshell.com>
-  **Docs** **GNU Bash Reference Manual Oficial**
<https://www.gnu.org/software/bash/manual/>

>_ Snippet Útil: Escaneo Rápido de Hosts Vía Ping en Bash

```
#!/usr/bin/env bash
# Sweep rápido de red /24 en paralelo usando subshells
SUBNET="10.10.10"
echo "[*] Escaneando subred ${SUBNET}.0/24..."

for ip in $(seq 1 254); do
    (ping -c 1 -W 1 "${SUBNET}.${ip}" &>/dev/null && echo "[+] Host activo: ${SUBNET}.${ip}") &
done
wait
echo "[*] Barrido completado."
```

1.3. Técnicas de Explotación & Seguridad Ofensiva

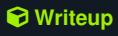
En esta sección se correlacionan las técnicas de intrusión con los retos o laboratorios resueltos dentro de este mismo documento.

1.3.1. Explotación de Servicios de Red: Samba Username Map Script

Mecanismo de Vulnerabilidad (CVE-2007-2447)

En versiones vulnerables de Samba (3,0,0 a 3,0,25rc3), la directiva `username map script` ejecutaba scripts de shell externos concatenando directamente el nombre de usuario sin escapar metacaracteres (‘ o \$(...)). Al pasar un payload como nombre de usuario vía SMB, el comando se ejecuta con privilegios de root.

Fuentes y Referencias del Concepto

-  **HackTheBox — Lame (Capítulo ??)**
[Máquina donde se identificó y explotó manualmente este vector en la práctica](#)
-  **NIST NVD — Registro Oficial CVE-2007-2447**
<https://nvd.nist.gov/vuln/detail/CVE-2007-2447>
-  **Rapid7 Metasploit Module Documentation**
https://www.rapid7.com/db/modules/exploit/multi/samba/usermap_script/

1.3.2. Explotación de Binarios: Format String Vulnerability

Fuga Arbitraria de Memoria del Stack

Cuando una función variádica como `printf()` recibe una entrada del usuario sin una cadena de formato estricta (`printf(buf)` en lugar de `printf("%s", buf)`), el usuario puede suministrar especificadores como `%p` o `%x` para leer valores del stack, o `%n` para escribir en memoria. En `x86_64` es fundamental considerar el orden Little-Endian al reconstruir cadenas desde valores hexadecimales.

Fuentes y Referencias del Concepto

-  **picoCTF — Format String 1 (Capítulo ??)**
[Reto donde se aplicó extracción de datos posicionales `%14\$p` para leer la bandera secreta](#)
-  **Hacking: The Art of Exploitation (Jon Erickson)**
[Capítulo 0x300: *Exploitation — Format Strings*](#)
-  **OWASP Format String Vulnerability Reference**
https://owasp.org/www-community/attacks/Format_string_attack

1.4. Matriz Global de Aprendizajes & Fuentes

Esta tabla proporciona un índice rápido para localizar de inmediato qué técnica domina el auditor y dónde consultar la referencia teórica o práctica correspondiente:

Técnica / Concepto	Categoría	Fuente Principal (Libro / Web / Lab)	Referencia
Llamadas al sistema y registros en x86_64	Arquitectura ASM	Libro Web <i>Practical Binary Analysis</i> Brown Univ. Cheatsheet	Cap. 1
Subshells y Descriptores de Ficheros	Scripting / Bash	Libro Docs <i>The Linux Command Line</i> GNU Bash Manual	Cap. 1
Inyección de comandos vía Samba	Redes / RCE	Writeup Docs Máquina Lame (HTB) CVE-2007-2447	Sec. ??
Format String Stack Leak (%p)	Binary Exploitation	Lab Libro Reto Format String 1 (picoCTF) <i>Hacking: The Art of Expl.</i>	Sec. ??

Tabla 1.1: Matriz consolidada de conocimientos adquiridos y fuentes de procedencia.